

Smart GST Analytics & Compliance Assistant

MERN Stack Frontend + Backend + MongoDB Complete Setup Guide

1. PROJECT STRUCTURE

```
smart-gst-assistant/  
├── client/           # React Frontend  
├── server/           # Node + Express Backend  
├── ml-service/       # Python ML Service  
├── datasets/  
├── docs/  
└── README.md
```

2. TECH STACK

Layer	Technology
Frontend	React.js + Tailwind CSS
Backend	Node.js + Express.js
Database	MongoDB Atlas
Authentication	JWT
Charts	Recharts / Chart.js
File Upload	Multer
ML Service	Flask / FastAPI
Deployment	Vercel + Render

3. FRONTEND GENERATION PROMPT

Use this prompt in:

- ChatGPT
- Claude
- Cursor AI

- Windsurf
 - Lovable
 - Bolt.new
-

FRONTEND MASTER PROMPT

Create a modern professional MERN stack frontend for a project called "Smart GST Analytics & Compliance Assistant".

Requirements:

1. Tech Stack:

- React.js
- Tailwind CSS
- React Router DOM
- Axios
- Recharts
- React Hook Form

2. Theme:

- Professional fintech dashboard
- Indian GST/business theme
- Responsive design
- Clean UI
- Modern cards and charts

3. Pages Required:

Authentication:

- Login page
- Signup page

Dashboard:

- GST summary cards
- Monthly GST trend graph
- Input tax vs Output tax graph
- Vendor contribution chart
- Recent invoices table

Invoice Management:

- Add invoice form
- Invoice listing table
- Edit invoice
- Delete invoice
- Search and filter invoices

Analytics Module:

- Monthly tax trends
- Seasonal analysis

- Vendor analytics
- Tax distribution charts

Prediction Module:

- Predicted GST liability card
- Forecast graph

Anomaly Detection Module:

- Suspicious invoices section
- Duplicate invoice alerts
- High transaction alerts

Reports Module:

- Generate PDF button
- Export CSV button

4. Components Required:

- Sidebar navigation
- Navbar
- Dashboard cards
- Tables
- Charts
- Modals
- Forms
- Notifications

5. Design Instructions:

- Use Tailwind CSS
- Use responsive grid layouts
- Use smooth hover effects
- Use professional typography
- Use dashboard-style UI

6. Backend Integration:

- Create Axios API service layer
- Use JWT token authentication
- Use protected routes

7. Folder Structure:

Create scalable frontend architecture with:

- pages/
- components/
- services/
- routes/
- context/
- hooks/
- utils/

8. Generate:

- Full frontend structure
- React components

- Routing setup
- API integration structure
- State management setup
- Dashboard UI

Do not generate dummy explanations.
Generate production-ready React frontend code.

4. BACKEND GENERATION PROMPT

BACKEND MASTER PROMPT

Create a scalable backend for a MERN stack project called "Smart GST Analytics & Compliance Assistant".

Tech Stack:

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT Authentication

Requirements:

1. Folder Structure:

- controllers/
- models/
- routes/
- middleware/
- services/
- config/
- utils/

2. Features:

Authentication:

- Register user
- Login user
- JWT token generation
- Password hashing using bcrypt

Invoice Module:

- Add invoice
- Update invoice
- Delete invoice
- Fetch invoices
- Search invoices

GST Engine:

- Calculate CGST
- Calculate SGST
- Calculate IGST
- Input Tax Credit calculation
- Net GST payable calculation

Analytics APIs:

- Monthly GST summary
- Vendor contribution
- Tax trend analytics
- Seasonal trend analytics

Prediction APIs:

- Fetch GST prediction from ML service

Anomaly APIs:

- Detect suspicious invoices
- Duplicate invoice detection

Reports APIs:

- Generate GST summary reports
- Export CSV/PDF

3. Security:

- JWT middleware
- Input validation
- Environment variables
- Error handling middleware

4. Database:

Use MongoDB with Mongoose schemas.

5. Generate:

- Express server setup
- All routes
- Controllers
- Middleware
- MongoDB models
- API structure
- Error handling
- Validation
- Production-ready backend structure

5. HOW TO CREATE MONGODB DATABASE (STEP-BY-STEP)

STEP 1 — Create MongoDB Atlas Account

Website: <https://www.mongodb.com/atlas>

Steps: 1. Sign up using Google/GitHub 2. Create free cluster 3. Select FREE tier 4. Choose nearest region 5. Create cluster

STEP 2 — Create Database User

Inside Atlas:

1. Go to Database Access
2. Add new database user
3. Set username
4. Set password
5. Give Read and Write permissions

Save username/password.

STEP 3 — Allow Network Access

1. Go to Network Access
2. Add IP Address
3. Choose:

Allow Access From Anywhere

STEP 4 — Get MongoDB Connection URL

1. Click Connect
2. Choose Drivers
3. Copy connection string

Example:

mongodb+srv://username:password@cluster.mongodb.net/smartgst

STEP 5 — Create Backend Project

Commands:

```
mkdir server
cd server
npm init -y
```

STEP 6 — Install Backend Dependencies

```
npm install express mongoose dotenv cors bcryptjs jsonwebtoken multer
```

Development Dependency:

```
npm install nodemon --save-dev
```

STEP 7 — Create Backend Structure

```
server/
|
├─ config/
├─ controllers/
├─ middleware/
├─ models/
├─ routes/
├─ utils/
├─ server.js
└─ .env
```

STEP 8 — Create .env File

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
```

STEP 9 — CONNECT MONGODB

config/db.js

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB Connected');
  } catch (error) {
    console.log(error);
    process.exit(1);
  }
};

module.exports = connectDB;
```

STEP 10 — CREATE MONGOOSE MODELS

User Model

models/User.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  password: {
    type: String,
    required: true
  },
  gstin: {
    type: String,
    required: true
  }
});
```



```
}, {
  timestamps: true
});

module.exports = mongoose.model('User', userSchema);
```

Invoice Model

models/Invoice.js

```
const mongoose = require('mongoose');

const invoiceSchema = new mongoose.Schema({
  invoiceNumber: {
    type: String,
    required: true
  },
  sellerGSTIN: {
    type: String,
    required: true
  },
  buyerGSTIN: {
    type: String,
    required: true
  },
  amount: {
    type: Number,
    required: true
  },
  gstRate: {
    type: Number,
    required: true
  },
  gstType: {
    type: String,
    enum: ['CGST_SGST', 'IGST'],
    required: true
  },
  taxAmount: {
    type: Number,
    required: true
  },
  state: {
    type: String,
    required: true
  },
  vendorName: {
```

```
        type: String
      },
      date: {
        type: Date,
        default: Date.now
      }
    }, {
      timestamps: true
    });

module.exports = mongoose.model('Invoice', invoiceSchema);
```

Tax Summary Model

models/TaxSummary.js

```
const mongoose = require('mongoose');

const taxSummarySchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  },
  totalInputTax: Number,
  totalOutputTax: Number,
  netGSTPayable: Number,
  month: String,
  year: Number
}, {
  timestamps: true
});

module.exports = mongoose.model('TaxSummary', taxSummarySchema);
```

STEP 11 — CREATE EXPRESS SERVER

server.js

```
const express = require('express');
const dotenv = require('dotenv');
const cors = require('cors');
const connectDB = require('./config/db');
```

```
dotenv.config();
connectDB();

const app = express();

app.use(cors());
app.use(express.json());

app.get('/', (req, res) => {
  res.send('Smart GST Backend Running');
});

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

STEP 12 — START SERVER

Add script in package.json:

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

Run:

```
npm run dev
```

6. FRONTEND SETUP

STEP 1 — Create React App

```
npx create-react-app client
```

OR

```
npm create vite@latest
```

STEP 2 — Install Frontend Libraries

```
npm install react-router-dom axios recharts react-hook-form
```

Tailwind CSS:

```
npm install -D tailwindcss postcss autoprefixer  
npx tailwindcss init -p
```

STEP 3 — Frontend Folder Structure

```
client/  
|  
├─ src/  
|   ├─ components/  
|   ├─ pages/  
|   ├─ services/  
|   ├─ routes/  
|   ├─ context/  
|   ├─ hooks/  
|   ├─ utils/  
|   └─ App.jsx
```

7. API TESTING

Use:

- Postman
- Thunder Client VS Code Extension

Test:

- Login
- Register
- Add invoice
- GST calculations
- Analytics

8. HOW TO CONNECT FRONTEND + BACKEND

Frontend Axios Base URL:

```
http://localhost:5000/api
```

Use:

- Axios interceptors
- JWT token headers

9. ML SERVICE SETUP

Create:

```
ml-service/
```

Install:

```
pip install flask pandas scikit-learn joblib
```

Features:

- GST prediction
- Anomaly detection
- EDA processing

10. DEPLOYMENT

Frontend:

- Vercel

Backend:

- Render

Database:

- MongoDB Atlas

ML API:

- Render
-

11. RECOMMENDED DEVELOPMENT ORDER

PHASE 1

- Authentication
- Database
- Invoice CRUD

PHASE 2

- GST engine
- Dashboard
- Charts

PHASE 3

- EDA
- Prediction
- Anomaly detection

PHASE 4

- Reports
 - Deployment
 - Optimization
-

12. BONUS FEATURES

Optional:

- OCR Invoice Scanner
 - AI Tax Assistant Chatbot
 - Email Notifications
 - GST Filing Reminder
 - Multi-user roles
 - Admin Dashboard
-

13. FINAL ADVICE

Focus heavily on:

1. Clean dashboard
2. Real GST workflow
3. Good charts
4. ML-based anomaly detection
5. Proper authentication
6. Responsive UI
7. Professional presentation

These are the features that will impress:

- Interviewers
- Faculty
- Judges
- Recruiters